Лабораторная работа № 2

Тема: Первые модульные тесты с Jest

Цель: Освоить запуск Jest и проверку функции calculateOrderTotal() без базы данных.

Дисциплина: Node.js и Express.js. РО 4.5

Что такое Jest?

Jest — инструмент для автоматического тестирования JavaScript. Он запускает тесты и сравнивает фактический результат программы с ожидаемым. В ЛР-1 использовали встроенный тестировщик node:test; здесь освоим Jest.

> **Перед началом**
>
> В проекте уже должен существовать файл services/orderMath.js из ЛР-1 с функцией calculateOrderTotal(). Не переписывайте его и не удаляйте tests/orderMath.test.js. База данных и сервер Express для этой лабораторной не нужны.

1. Проверьте структуру проекта

```
leson8prisma/
  services/
    orderMath.js                ← уже существует
  tests/
    orderMath.test.js           ← ЛР-1, оставить без изменений
    unit/
      orderMath.test.js         ← создадим для Jest
  jest.config.js                ← создадим
  package.json                  ← дополним команду test
```

2. Установите Jest

Откройте PowerShell в каталоге C:\WORK\js\leson8prisma и выполните:

```
npm install --save-dev jest
```

Команда устанавливает Jest как зависимость для разработки. Повторная установка, если пакет уже установлен, не требуется.

3. Настройте команду запуска

Откройте package.json. Найдите существующий объект scripts и добавьте в него поле test. Существующие start и другие команды НЕ удаляйте.

```
"scripts": {
  "start": "tsx app.js",
  "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js"
}
```

Это пример фрагмента package.json. Значение start сохраните таким, как оно уже записано в вашем проекте. Не создавайте второй объект scripts.

4. Изолируйте Jest от тестов ЛР-1

В корне проекта создайте (или замените содержимое) файл jest.config.js следующим кодом:

```
export default {
  testEnvironment: "node",
  transform: {},
  testMatch: ["**/tests/unit/orderMath.test.js"]
};
```

testMatch указывает Jest запускать только tests/unit/orderMath.test.js. Поэтому Jest не запускает ни старые тесты node:test из ЛР-1, ни файлы productService.test.js из прежней сложной версии ЛР-2. Ничего удалять не нужно.

5. Создайте файл с тестами

Создайте файл C:\WORK\js\leson8prisma\tests\unit\orderMath.test.js. Если файл уже есть, замените его содержимое следующим полным кодом:

```
import { describe, expect, test } from "@jest/globals";
import { calculateOrderTotal } from "../../services/orderMath.js";

describe("Расчёт суммы заказа", () => {
  test("сумма двух товаров", () => {
    const items = [
      { price: 10000, quantity: 2 },
      { price: 5000, quantity: 1 }
    ];
    const total = calculateOrderTotal(items);
    expect(total).toBe(25000);
  });

  test("пустой заказ — сумма 0", () => {
    expect(calculateOrderTotal([])).toBe(0);
  });

  test("количество 0 — ошибка", () => {
    expect(() => calculateOrderTotal([
      { price: 1000, quantity: 0 }
    ])).toThrow(/quantity/);
  });

  // Самостоятельное задание: добавьте тест для quantity = 101.
});
```

6. Разберём три проверки

Тест 1: для двух товаров 10000 × 2 + 5000 × 1 = 25000. Метод toBe(25000) сравнивает ожидаемую сумму с полученной.

Тест 2: пустой массив товаров [] должен давать сумму 0.

Тест 3: quantity = 0 недопустимо. toThrow(/quantity/) проверяет, что функция выбрасывает ошибку с текстом quantity. Это отрицательный тест.

7. Запустите Jest и проверьте результат

В терминале проекта выполните:

```
npm test
```

```

```

Если все файлы сохранены и функция calculateOrderTotal() соответствует ЛР-1, Jest должен сообщить о трёх успешно пройденных тестах. Проверка в базе данных не выполняется.

```
PASS tests/unit/orderMath.test.js
Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
```

Предупреждение ExperimentalWarning о VM Modules возможно при работе Jest с ES-модулями и само по себе не означает ошибку теста.

8. Посмотрите, как Jest обнаруживает ошибку

В тесте «сумма двух товаров» временно замените expect(total).toBe(25000) на expect(total).toBe(25001). Снова выполните npm test.

Ожидается FAIL: Jest покажет, что ожидалось 25001, а получено 25000. Верните 25000 и повторно запустите npm test — снова должен быть PASS.

9. Самостоятельное задание

В файл tests/unit/orderMath.test.js внутри describe(), ПЕРЕД последней строкой }); добавьте ещё один тест: при quantity = 101 функция должна выбросить ошибку. По аналогии используйте toThrow(/quantity/).

После добавления выполните npm test. При правильном выполнении будут успешно пройдены уже четыре теста.

> **Важно**
>
> В самостоятельном задании НЕ нужно изменять services/orderMath.js, создавать методы сервиса или новые таблицы. Все тесты обязательной части должны проходить до выполнения самостоятельного задания.

10. Что сдавать

1. Файл jest.config.js с ограничением testMatch.
2. Файл tests/unit/orderMath.test.js с четырьмя тестами (три основных и один самостоятельный).
3. Скриншот терминала с успешным запуском npm test.

Контрольные вопросы

4. Для чего нужен Jest?
5. Что сравнивает expect(total).toBe(25000)?
6. Почему Jest не запускает тесты ЛР-1?
7. Что означает PASS и FAIL?
8. Зачем нужен отрицательный тест с toThrow()?

Критерии оценивания

Критерий	Баллы
Jest установлен и запускается	20
jest.config.js изолирует старые тесты	20

Критерий	Баллы
Три основных теста проходят	30
Самостоятельный тест quantity = 101	20
Студент объясняет результаты	10